



SYSTEME DE VENTE DES LIVRES

(Mise en place d'un pipeline avec Streamlit – Postgres - Airflow – DBT –
Snowflake – Power BI)

Présenté par :

MOUTSITA Rovincy-Prince-Lémy

BANIEK Eudes Exaucé

YABI Nadine Rémi

Etudiants Master II IA - DIT

Cours : Technologie Big Data

Date de soumission : 30 avril 2026

INTRODUCTION

Le rythme croissant auquel le monde se développe pousse les acteurs de cette société moderne à prendre des mesures nécessaires afin de suivre des processus normaux de croissances sans faiblir bien que d'autres chancellent. La donnée aujourd'hui se veut plurielle, sous différents formats à savoir texte, son, vidéo et bien d'autres. D'où la nécessité de mettre en place des mécanismes de suivi des processus sans jamais casser l'ensemble des programmes. La donnée est partout, elle régit le nouvel ordre mondial et de sa qualité dépend l'avenir à la fois des grands comme des petits groupes. C'est dans cette perspective afin d'être en phase avec les besoins présent du marché, il nous a été soumis ce travail, la mise en place d'un pipeline avec les technologies modernes dont DBT et Snowflake en vue gérer le processus de traitement de la donnée via un flux ELT (Extract, Load, Transform) car nous chargeons les données dans Snowflake avant de transformer avec DBT. Dans les lignes qui vont suivre, nous expliquerons ce qui a été fait et pourquoi avant préférer conteneuriser notre travail ainsi que les résultats attendus, tout en expliquant la démarche à suivre.

MISE EN PLACE DES PROCESS

Tout d'abord, pour devoir réaliser ou lancer ce programme, l'utilisateur doit se rassurer que docker soit installé sur son laptop et posséder un compte GitHub pour effectuer le clonage de notre repos, dont voici le lien :

https://github.com/Moutsita/exam_tech_bigdata.git

Tout d'abord, les membres ayant contribués à la réalisation du projet ont tapé la commande suivante en local, dans un dossier ou disque de leur choix pour réaliser le clonage :

```
git clone https://github.com/Moutsita/exam_tech_bigdata.git
```

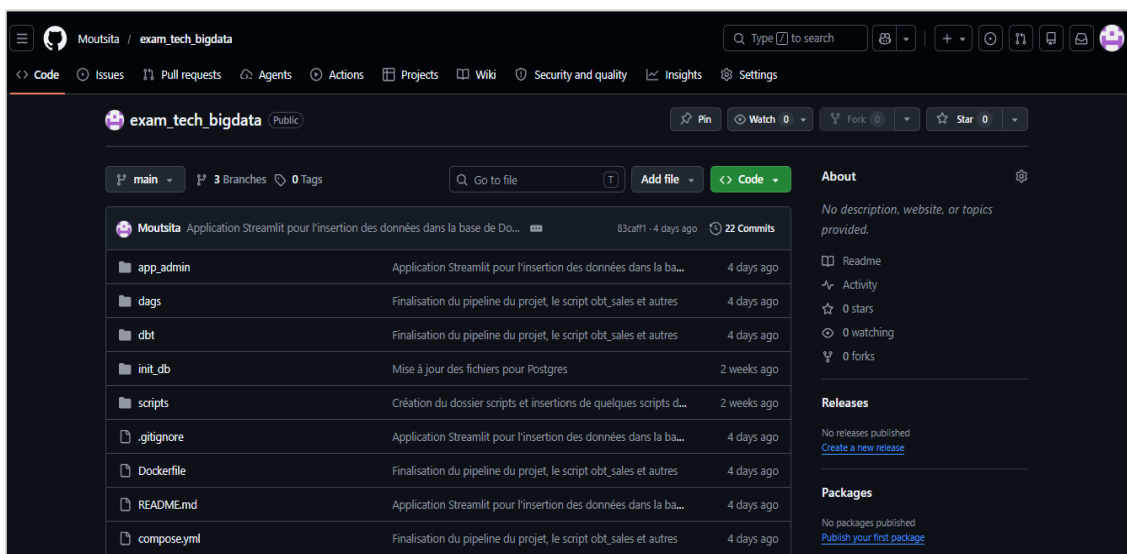


Fig1.a : Illustration de notre dépôt distant

Et dont voici la liste des contributeurs :

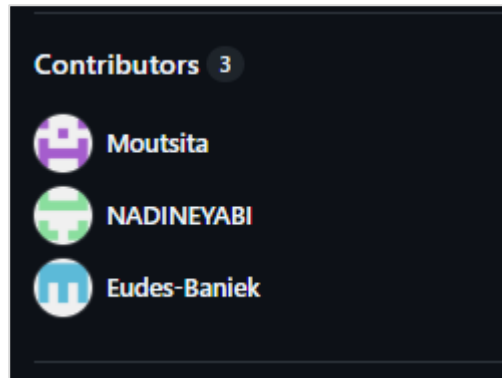


Fig1.b : Liste des contributeurs du projet

Etape 0 : Pourquoi avons-nous voulu la conteneurisation ?

En effet, il est judicieux pour nous car nous gardons tous le même modèle, les mêmes versions et configurations qu'importe l'OS utilisé, cela garantit la stabilité du programme et nous évite les problèmes de cassures de version, induisant parfois à la rétrogradation à une version pour s'assurer que le programme tourne et fonctionne sur notre machine.

Etape 1 : Lancement du programme

Elle s'effectue avec la commande, dans un terminal au niveau du lien faisant référence à notre dépôt :

```
docker compose up -d
```

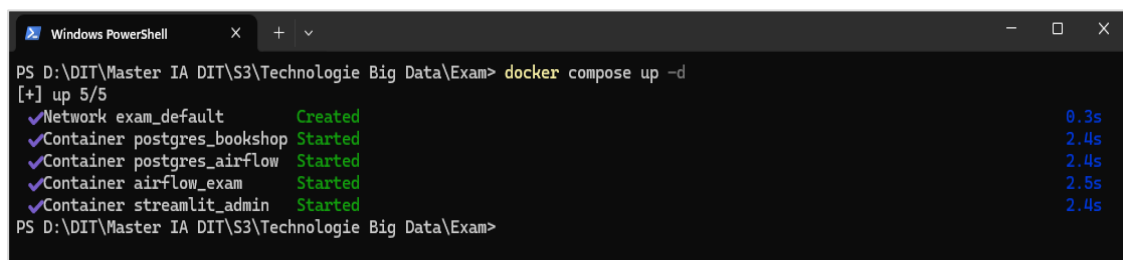


Fig1.c : Illustration de la commande ci-dessus

Etape 2 : Configuration de airflow

Pour se connecter à l'orchestrateur airflow, aller à l'url suivante :

<http://localhost:8080>

Ensuite, mettre les identifiants de connexion comme suit :

Login : admin password : admin123

Il renverra vers la page de airflow

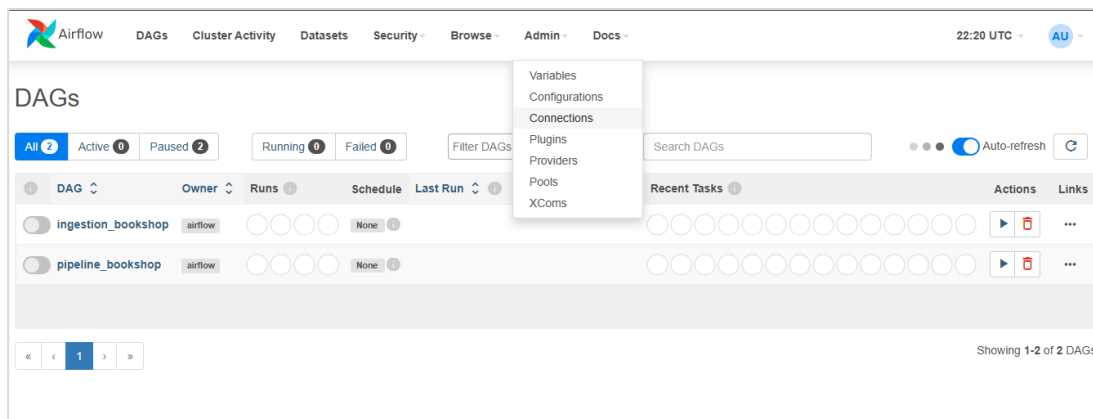


Fig1.d : Page d'accueil de airflow

Une fois dans la page d'accueil, aller à l'onglet '**admin**' et cliquer sur '**connections**' afin de procéder à la configuration de airflow à notre base de données postgres et ensuite un clic final sur le bouton '**save**' pour enregistrer toutes les modifications.

Fig1.e : Configuration de postgres dans le container airflow

NB : Pour les informations de configurations voir le README.md (le point 8)

Etape 3 : Création de la base de données et des tables dans snowflake

Pour cette partie, que de chercher à réécrire continuellement le script dans Snowflake, nous avons procédé à la création de trois scripts dont nous ne garderons que deux pour l'exécution du programme. Les présents scripts se trouve dans le sous dossier '**scripts**' et le nom de chaque fichier commence par '**snow_***'.

Ils ont permis se rassurer que notre création de tables ne casse pas.

Par la suite dans snowflake, nous avons cliqués sur '**projects**' et il nous ramène dans '**my Workspace**' et on clique sur '**Add new**' et on sélectionne '**Folder**' et on lui attribut un nom, dans notre cas : '**exam_tech_bigdata**'.

A partir de ce dossier, on n'importe nos fichiers du dossier '**scripts**'.

Le script '**snow_insertions_donnees.sql**' n'est exécuté que pour s'assurer que l'insertion peut se faire dans nos tables. S'il est exécuté, ouvrir notre script '**snow_requetes.sql**' et exécuter la commande suivante :

```
DROP DATABASE IF EXISTS BOOKSHOP;
```

Il supprimera notre base de données et son contenu et à partir de là, on n'exécute qu'un seul script : '**snow_postgres_diagram_bigdata_m2.sql**'

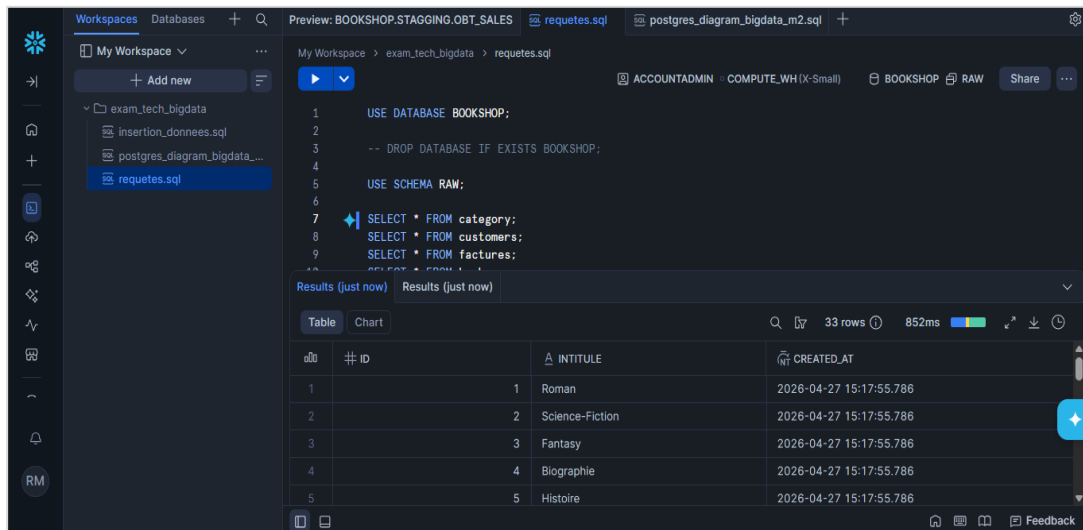


Fig1.f : Illustration de nos scripts importés dans snowflake

Etape 4 : Vérification de la création des données dans la BD Postgres

Elle se fait à travers les commandes suivantes :

Accès au terminal du container de la BD

```
docker exec -it postgres_bookshop bash
```

Accès à la base de données

```
psql -U user -d bookshop
```

Lister les bases de données

```
\l
```

Lister les schémas

```
\dn+
```

```

PS D:\DIT\Master IA DIT\S3\Technologie Big Data\Exam> docker exec -it postgres_bookshop bash
root@9df9fb012eac:/# psql -U user -d bookshop
psql (17.9 (Debian 17.9-1.pgdg13+1))
Type "help" for help.

bookshop=# \l

          List of databases
  Name | Owner | Encoding | Locale Provider | Collate | Ctype | Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
bookshop | user | UTF8 | libc | en_US.utf8 | en_US.utf8 | | | =c/user
postgres | user | UTF8 | libc | en_US.utf8 | en_US.utf8 | | | =c/user
template0 | user | UTF8 | libc | en_US.utf8 | en_US.utf8 | | | =c/user
template1 | user | UTF8 | libc | en_US.utf8 | en_US.utf8 | | | =c/user
(4 rows)

bookshop=# \dn+

          List of schemas
  Name | Owner | Access privileges | Description
-----+-----+-----+-----
marts | user | | 
public | pg_database_owner | pg_database_owner=UC/pg_database_owner+=U/pg_database_owner | standard public schema
raw | user | | 
staging | user | | 
warehouse | user | | 
(5 rows)

bookshop=#

```

Fig1.4. a : Illustration d'exécution des commandes ci-dessus

Lister les tables de notre base de données

```
\dt raw.*
```

Pour connaître la structure d'une table

```
\d raw.category # Affiche la structure de la table category
```

```
\d raw.* # Affichic la structure de toutes les tables
```

```

bookshop=# \dt raw.*

          List of relations
 Schema | Name      | Type  | Owner
-----+-----+-----+-----
raw     | books     | table | user
raw     | category  | table | user
raw     | customers | table | user
raw     | factures  | table | user
raw     | ventes    | table | user
(5 rows)

bookshop=# \d raw.category

          Table "raw.category"
  Column | Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
id       | integer       |           | not null | nextval('raw.category_id_seq'::regclass)
intitule | character varying(100) |           |          | 
created_at | timestamp without time zone |           |          | CURRENT_TIMESTAMP
Indexes:
    "category_pkey" PRIMARY KEY, btree (id)
Referenced by:
    TABLE "raw.books" CONSTRAINT "books_category_id_fkey" FOREIGN KEY (category_id) REFERENCES raw.category(id)

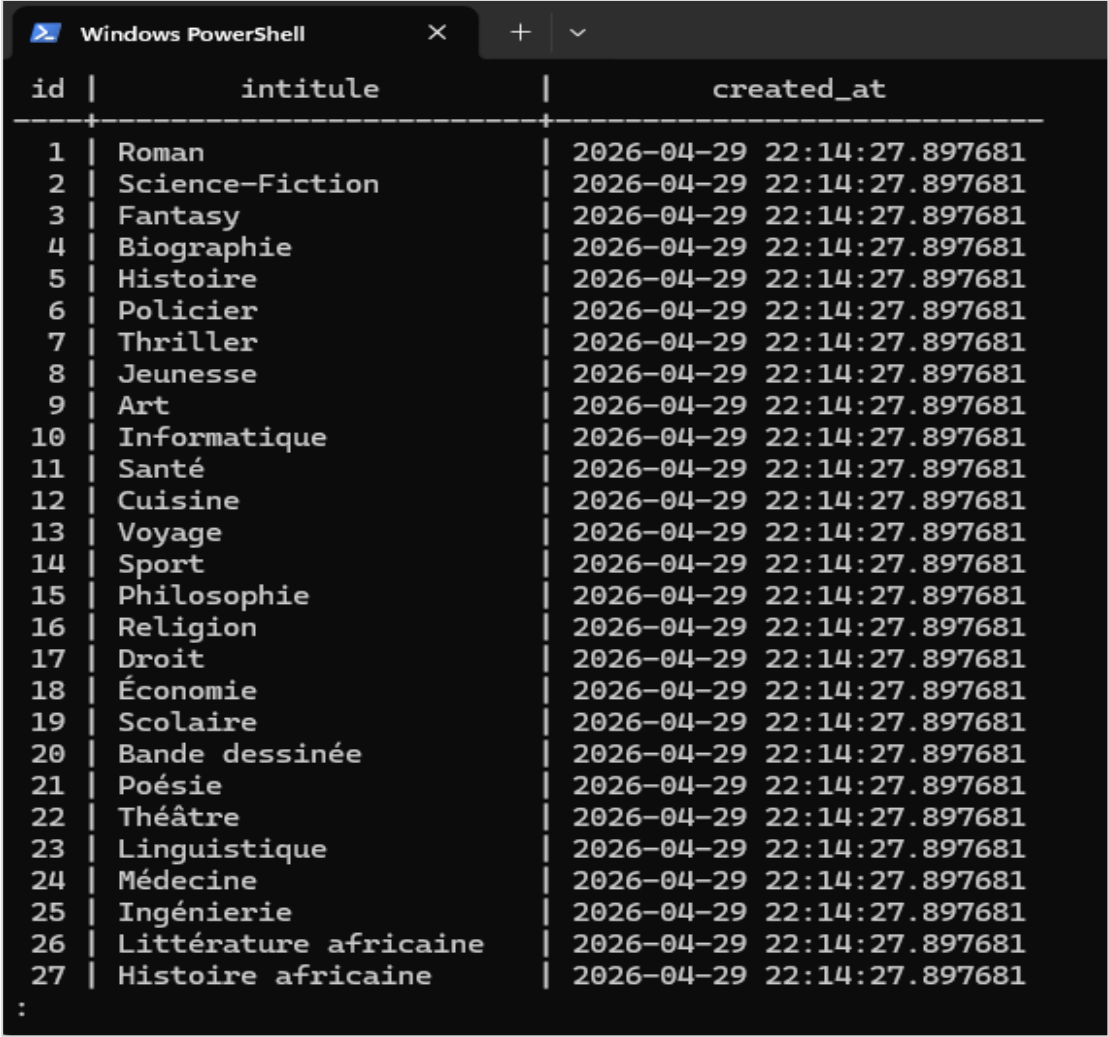
bookshop=#

```

Fig1.4. b : Illustration des deux premières commandes

Afficher les données d'une table, ici on affiche celle de category

```
SELECT * FROM raw.category ;
```



id	intitule	created_at
1	Roman	2026-04-29 22:14:27.897681
2	Science-Fiction	2026-04-29 22:14:27.897681
3	Fantasy	2026-04-29 22:14:27.897681
4	Biographie	2026-04-29 22:14:27.897681
5	Histoire	2026-04-29 22:14:27.897681
6	Policier	2026-04-29 22:14:27.897681
7	Thriller	2026-04-29 22:14:27.897681
8	Jeunesse	2026-04-29 22:14:27.897681
9	Art	2026-04-29 22:14:27.897681
10	Informatique	2026-04-29 22:14:27.897681
11	Santé	2026-04-29 22:14:27.897681
12	Cuisine	2026-04-29 22:14:27.897681
13	Voyage	2026-04-29 22:14:27.897681
14	Sport	2026-04-29 22:14:27.897681
15	Philosophie	2026-04-29 22:14:27.897681
16	Religion	2026-04-29 22:14:27.897681
17	Droit	2026-04-29 22:14:27.897681
18	Économie	2026-04-29 22:14:27.897681
19	Scolaire	2026-04-29 22:14:27.897681
20	Bande dessinée	2026-04-29 22:14:27.897681
21	Poésie	2026-04-29 22:14:27.897681
22	Théâtre	2026-04-29 22:14:27.897681
23	Linguistique	2026-04-29 22:14:27.897681
24	Médecine	2026-04-29 22:14:27.897681
25	Ingénierie	2026-04-29 22:14:27.897681
26	Littérature africaine	2026-04-29 22:14:27.897681
27	Histoire africaine	2026-04-29 22:14:27.897681

Fig1.4. c : Sortie de commande ci-dessus

Pour quitter

```
\q
```

Et enfin **'exit'** pour revenir à notre shell.

N.B : Toutes ces commandes peuvent s'exécuter pour interroger le reste des tables.

Etape 5 : Création du fichier profiles.yml dans le container

Au premier lancement des configurations, ce fichier se crée automatiquement, via la commande suivante :

```
docker exec -it airflow_exam bash -c "cd /opt/airflow/dbt && dbt init exam_tech_bigdata"
```

Il vous demandera de faire les configurations nécessaires afin de pouvoir accéder à snowflake (voir le point 11 du README.md pour le model de configuration).

Cependant lorsque, le programme est arrêté, il disparaît car il est un fichier éphémère et ne figure pas dans notre dépôt local pour des raisons de sécurité car il peut arriver qu'il soit pusher par mégarde.

C'est pourquoi, les commandes qui vont suivre sont palliatives à cette ambiguïté :

Accès au terminal de airflow

```
docker exec -it airflow_exam bash
```

Ensuite se positionner sur ce chemin d'accès

```
cd /opt/airflow/dbt/exam_tech_bigdata
```

Et enfin, exécuter la commande suite

```
mkdir -p ~/.dbt/ && cat << EOF > ~/.dbt/profiles.yml
exam_tech_bigdata:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: [ton_id_snowflake]
      user: [ton_utilisateur]
      password: [ton_mot_de_passe]
      role: ACCOUNTADMIN
      database: BOOKSHOP
      warehouse: COMPUTE_WH
      schema: STAGGING
      threads: 4
EOF
```

N.B : Remplacer ces points par ces identifiants Snowflake. Ces commandes sont évoqués dans le README.md (voir le point 8).

Par suite, afin de se rassurer que le fichier profiles.yml est bien créé, on exécute la commande suivante :

```
cat ~/.dbt/profiles.yml
```

Il devrait nous faire ressortir nos informations ce qui a été inséré dans notre fichier comme, l'illustre l'image ci-dessous :


```

PS D:\DIT\Master IA DIT\S3\Technologie Big Data\Exam> docker exec -it airflow_exam bash
airflow@be1b7f3f285a:/opt/airflow$ cd /opt/airflow/dbt/exam_tech_bigdata
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$ mkdir -p ~/.dbt/ && cat << EOF > ~/.dbt/profiles.yml
exam_tech_bigdata:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: [ton_id_snowflake]
      user: [ton_utilisateur]
      password: [ton_mot_de_passe]
      role: ACCOUNTADMIN
      database: BOOKSHOP
      warehouse: COMPUTE_WH
      schema: STAGGING
      threads: 4
EOF
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$ cat ~/.dbt/profiles.yml
exam_tech_bigdata:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: [ton_id_snowflake]
      user: [ton_utilisateur]
      password: [ton_mot_de_passe]
      role: ACCOUNTADMIN
      database: BOOKSHOP
      warehouse: COMPUTE_WH
      schema: STAGGING
      threads: 4
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$ |

```

Fig1.5 : Création de profiles.yml après redémarrage du programme

Remarque : Pour des raisons de sécurité, nous n'avons pas pris nos véritables identifiants de connexion à snowflake ; ne pas oublier de les remplacer.

Etape 6 : Configuration et test de DBT

Pour ce faire, nous procéderons de la manière suivante, en tapant ces commandes dans cet ordre :

```
dbt debug
```

Il permet de vérifier si tous notre programme contient des erreurs de configurations, ou un fichier manquant pouvant empêcher le démarrage du programme.

Cette commande est dans le README.md (voir le point 8)

```
airflow@be1b7f3f285a: /opt/airflow/dbt/exam_tech_bigdata$ dbt debug
01:48:21 Running with dbt=1.8.7
01:48:21 dbt version: 1.8.7
01:48:21 python version: 3.8.18
01:48:21 python path: /usr/local/bin/python
01:48:21 os info: Linux-6.6.87.2-microsoft-standard-WSL2-x86_64-with-glibc2.34
01:48:23 Using profiles dir at /home/airflow/.dbt
01:48:23 Using profiles.yml file at /home/airflow/.dbt/profiles.yml
01:48:23 Using dbt_project.yml file at /opt/airflow/dbt/exam_tech_bigdata/dbt_project.yml
01:48:23 adapter type: snowflake
01:48:23 adapter version: 1.8.4
01:48:23 Configuration:
01:48:23   profiles.yml file [OK found and valid]
01:48:23   dbt_project.yml file [OK found and valid]
01:48:23 Required dependencies:
01:48:23   - git [OK found]

01:48:23 Connection:
01:48:23   account: [REDACTED]
01:48:23   user: [REDACTED]
01:48:23   database: BOOKSHOP
01:48:23   warehouse: COMPUTE_WH
01:48:23   role: ACCOUNTADMIN
01:48:23   schema: STAGGING
01:48:23   authenticator: None
01:48:23   oauth_client_id: None
01:48:23   query_tag: None
01:48:23   client_session_keep_alive: False
01:48:23   host: None
01:48:23   port: None
01:48:23   proxy_host: None
01:48:23   proxy_port: None
01:48:23   protocol: None
01:48:23   connect_retries: 1
01:48:23   connect_timeout: None
01:48:23   retry_on_database_errors: False
01:48:23   retry_all: False
01:48:23   insecure_mode: False
01:48:23   reuse_connections: None
01:48:23 Registered adapter: snowflake=1.8.4
01:48:27 Connection test: [OK connection ok]

01:48:27 All checks passed!
airflow@be1b7f3f285a: /opt/airflow/dbt/exam_tech_bigdata$
```

Fig1.6. a : dbt debug qui montre que tout est OK

Une fois exécuté et que le retour est « **All checks passed** », on peut alors compiler le projet contenant nos models, pour ainsi être certains que nos flux de données peuvent enfin transiter de airflow-dbt vers snowflake.

Pour cela, la commande suivante permet de lancer la compilation :

```
dbt run
```

A la sortie, aucune erreur ; alors nos données peuvent bien migrer de la base de données (Postgres SQL) vers Snowflake (cloud) via Airflow-DBT par l'ingestion (« dbt run ») et transmis de Snowflake à DBT (pour les sous dossiers de « models ») qui permettent de faire des transformations et les renvoyer dans des schémas spécifiques créés pour cela.

```
airflow@be1b7f3f285a: /opt/airflow/dbt/exam_tech_bigdata$ dbt run
02:22:56 Running with dbt=1.8.7
02:22:58 Registered adapter: snowflake=1.8.4
02:22:59 Found 14 models, 28 data tests, 5 sources, 1 exposure, 454 macros
02:22:59 Concurrency: 4 threads (target='dev')
02:23:03
02:23:03 1 of 14 START sql view model STAGGING.stg_books ..... [RUN]
02:23:03 2 of 14 START sql view model STAGGING.stg_category ..... [RUN]
02:23:03 3 of 14 START sql view model STAGGING.stg_customers ..... [RUN]
02:23:03 4 of 14 START sql view model STAGGING.stg_factures ..... [RUN]
02:23:04 1 of 14 OK created sql view model STAGGING.stg_books ..... [SUCCESS 1 in 1.34s]
02:23:04 5 of 14 START sql view model STAGGING.stg_ventes ..... [RUN]
02:23:04 2 of 14 OK created sql view model STAGGING.stg_category ..... [SUCCESS 1 in 1.35s]
02:23:04 3 of 14 OK created sql view model STAGGING.stg_customers ..... [SUCCESS 1 in 1.35s]
02:23:04 4 of 14 OK created sql view model STAGGING.stg_factures ..... [SUCCESS 1 in 1.35s]
02:23:04 6 of 14 START sql view model STAGGING_WAREHOUSE.dim_books ..... [RUN]
02:23:04 7 of 14 START sql view model STAGGING_WAREHOUSE.dim_category ..... [RUN]
02:23:04 8 of 14 START sql view model STAGGING_WAREHOUSE.dim_customers ..... [RUN]
02:23:05 5 of 14 OK created sql view model STAGGING.stg_ventes ..... [SUCCESS 1 in 1.22s]
02:23:05 9 of 14 START sql view model STAGGING_WAREHOUSE.fact_factures ..... [RUN]
02:23:05 8 of 14 OK created sql view model STAGGING_WAREHOUSE.dim_customers ..... [SUCCESS 1 in 1.18s]
02:23:05 10 of 14 START sql view model STAGGING_WAREHOUSE.fact_books_annees ..... [RUN]
02:23:05 6 of 14 OK created sql view model STAGGING_WAREHOUSE.dim_books ..... [SUCCESS 1 in 1.26s]
02:23:05 11 of 14 START sql view model STAGGING_WAREHOUSE.fact_books_jour ..... [RUN]
02:23:05 7 of 14 OK created sql view model STAGGING_WAREHOUSE.dim_category ..... [SUCCESS 1 in 1.28s]
02:23:05 12 of 14 START sql view model STAGGING_WAREHOUSE.fact_books_mois ..... [RUN]
02:23:07 9 of 14 OK created sql view model STAGGING_WAREHOUSE.fact_factures ..... [SUCCESS 1 in 1.29s]
02:23:07 11 of 14 OK created sql view model STAGGING_WAREHOUSE.fact_books_jour ..... [SUCCESS 1 in 1.20s]
02:23:07 12 of 14 OK created sql view model STAGGING_WAREHOUSE.fact_books_mois ..... [SUCCESS 1 in 1.19s]
02:23:07 10 of 14 OK created sql view model STAGGING_WAREHOUSE.fact_books_annees ..... [SUCCESS 1 in 1.29s]
02:23:07 13 of 14 START sql view model STAGGING_WAREHOUSE.fact_ventes ..... [RUN]
02:23:08 13 of 14 OK created sql view model STAGGING_WAREHOUSE.fact_ventes ..... [SUCCESS 1 in 1.21s]
02:23:08 14 of 14 START sql table model STAGGING_MARTS.obt_sales ..... [RUN]
02:23:14 14 of 14 OK created sql table model STAGGING_MARTS.obt_sales ..... [SUCCESS 1 in 6.04s]
02:23:14
02:23:14 Finished running 13 view models, 1 table model in 0 hours 0 minutes and 14.84 seconds (14.84s).
02:23:15
02:23:15 Completed successfully
02:23:15
02:23:15 Done. PASS=14 WARN=0 ERROR=0 SKIP=0 TOTAL=14
airflow@be1b7f3f285a: /opt/airflow/dbt/exam_tech_bigdata$
```

Fig1.6. b : dbt run illustre le succès de la compilation avec un total de PASS=14 ERROR=0 SKIP=0

Elle illustre la réussite de la compilation avec 14 PASS et aucune erreur ou saut dans la compilation.

Enfin, le test final, il compile minutieusement et recherche la moindre anomalie pouvant interrompre ou faire planter le pipeline. Elle est une étape cruciale, car de sa sortie dépend la stabilité de notre pipeline ; elle s'effectue par la commande suivante :

```
dbt test
```

```
airflow@be1b7f3f285a: /opt/ε X + v
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$ dbt test
02:40:46 Running with dbt=1.8.7
02:40:47 Registered adapter: snowflake=1.8.4
02:40:49 Found 14 models, 28 data tests, 5 sources, 1 exposure, 454 macros
02:40:49
02:40:51 Concurrency: 4 threads (target='dev')
02:40:51
02:40:51 1 of 28 START test not_null_dim_books_category_id ..... [RUN]
02:40:51 2 of 28 START test not_null_dim_books_id ..... [RUN]
02:40:51 3 of 28 START test not_null_dim_category_id ..... [RUN]
02:40:51 4 of 28 START test not_null_dim_customers_id ..... [RUN]
02:40:53 3 of 28 PASS not_null_dim_category_id ..... [PASS in 1.42s]
02:40:53 5 of 28 START test not_null_fact_factures_customers_id ..... [RUN]
02:40:53 1 of 28 PASS not_null_dim_books_category_id ..... [PASS in 1.43s]
02:40:53 2 of 28 PASS not_null_dim_books_id ..... [PASS in 1.45s]
02:40:53 4 of 28 PASS not_null_dim_customers_id ..... [PASS in 1.45s]
02:40:53 6 of 28 START test not_null_fact_factures_id ..... [RUN]
02:40:53 7 of 28 START test not_null_fact_ventes_books_id ..... [RUN]
02:40:53 8 of 28 START test not_null_fact_ventes_factures_id ..... [RUN]
02:40:53 5 of 28 PASS not_null_fact_factures_customers_id ..... [PASS in 1.12s]
02:40:54 9 of 28 START test not_null_fact_ventes_id ..... [RUN]
02:40:54 7 of 28 PASS not_null_fact_ventes_books_id ..... [PASS in 1.09s]
02:40:54 10 of 28 START test not_null_stg_books_category_id ..... [RUN]
02:40:54 8 of 28 PASS not_null_fact_ventes_factures_id ..... [PASS in 1.11s]
02:40:54 11 of 28 START test not_null_stg_books_id ..... [RUN]
02:40:54 6 of 28 PASS not_null_fact_factures_id ..... [PASS in 1.16s]
02:40:54 12 of 28 START test not_null_stg_category_id ..... [RUN]
02:40:55 9 of 28 PASS not_null_fact_ventes_id ..... [PASS in 1.03s]
02:40:55 13 of 28 START test not_null_stg_customers_id ..... [RUN]
02:40:55 12 of 28 PASS not_null_stg_category_id ..... [PASS in 0.97s]
02:40:55 14 of 28 START test not_null_stg_factures_customers_id ..... [RUN]
02:40:55 10 of 28 PASS not_null_stg_books_category_id ..... [PASS in 1.04s]
02:40:55 15 of 28 START test not_null_stg_factures_id ..... [RUN]
02:40:55 11 of 28 PASS not_null_stg_books_id ..... [PASS in 1.07s]
02:40:55 16 of 28 START test not_null_stg_ventes_books_id ..... [RUN]
02:40:56 13 of 28 PASS not_null_stg_customers_id ..... [PASS in 1.01s]
02:40:56 17 of 28 START test not_null_stg_ventes_factures_id ..... [RUN]
02:40:56 14 of 28 PASS not_null_stg_factures_customers_id ..... [PASS in 1.04s]
```

Fig1.6. c.1 : dbt test, commande à l'entrée
La sortie illustre la robustesse de notre pipeline.

```
airflow@be1b7f3f285a: /opt/ε X + v
02:40:55 11 of 28 PASS not_null_stg_books_id ..... [PASS in 1.07s]
02:40:55 16 of 28 START test not_null_stg_ventes_books_id ..... [RUN]
02:40:56 13 of 28 PASS not_null_stg_customers_id ..... [PASS in 1.01s]
02:40:56 17 of 28 START test not_null_stg_ventes_factures_id ..... [RUN]
02:40:56 14 of 28 PASS not_null_stg_factures_customers_id ..... [PASS in 1.04s]
02:40:56 18 of 28 START test not_null_stg_ventes_id ..... [RUN]
02:40:56 15 of 28 PASS not_null_stg_factures_id ..... [PASS in 1.08s]
02:40:56 19 of 28 START test unique_dim_books_id ..... [RUN]
02:40:56 16 of 28 PASS not_null_stg_ventes_books_id ..... [PASS in 1.05s]
02:40:56 20 of 28 START test unique_dim_category_id ..... [RUN]
02:40:57 17 of 28 PASS not_null_stg_ventes_factures_id ..... [PASS in 1.02s]
02:40:57 21 of 28 START test unique_dim_customers_id ..... [RUN]
02:40:57 18 of 28 PASS not_null_stg_ventes_id ..... [PASS in 1.04s]
02:40:57 22 of 28 START test unique_fact_factures_id ..... [RUN]
02:40:57 19 of 28 PASS unique_dim_books_id ..... [PASS in 1.08s]
02:40:57 23 of 28 START test unique_fact_ventes_id ..... [RUN]
02:40:57 20 of 28 PASS unique_dim_category_id ..... [PASS in 1.17s]
02:40:58 24 of 28 START test unique_stg_books_id ..... [RUN]
02:40:58 21 of 28 PASS unique_dim_customers_id ..... [PASS in 1.10s]
02:40:58 25 of 28 START test unique_stg_category_id ..... [RUN]
02:40:58 22 of 28 PASS unique_fact_factures_id ..... [PASS in 1.05s]
02:40:58 26 of 28 START test unique_stg_customers_id ..... [RUN]
02:40:58 23 of 28 PASS unique_fact_ventes_id ..... [PASS in 1.09s]
02:40:58 27 of 28 START test unique_stg_factures_id ..... [RUN]
02:40:58 24 of 28 PASS unique_stg_books_id ..... [PASS in 1.12s]
02:40:58 28 of 28 START test unique_stg_ventes_id ..... [RUN]
02:40:59 26 of 28 PASS unique_stg_customers_id ..... [PASS in 0.99s]
02:40:59 25 of 28 PASS unique_stg_category_id ..... [PASS in 1.04s]
02:40:59 27 of 28 PASS unique_stg_factures_id ..... [PASS in 1.04s]
02:40:59 28 of 28 PASS unique_stg_ventes_id ..... [PASS in 1.04s]
02:40:59
02:40:59 Finished running 28 data tests in 0 hours 0 minutes and 10.57 seconds (10.57s).
02:40:59
02:40:59 Completed successfully
02:40:59
02:40:59 Done. PASS=28 WARN=0 ERROR=0 SKIP=0 TOTAL=28
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$
```

Fig1.6. c.2 : dbt test montre que le test final est un succès avec PASS=28 ERROR=0 SKIP=0

Notre pipeline de données est enfin solide, évite la cassure ou le plantage.

Pour aller plus, nous pouvons avec DBT, nous pouvons toujours sur le même chemin d'accès taper les commandes suites :

```
dbt docs generate (1)
```

La première (1) génère la documentation DBT essentielle pour notre pipeline, les liaisons et communications entre nos différents schémas et requêtes adressées.

```
airflow@be1b7f3f285a: /opt/ε × + v
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$ dbt docs generate
03:03:34 Running with dbt=1.8.7
03:03:35 Registered adapter: snowflake=1.8.4
03:03:36 Found 14 models, 28 data tests, 5 sources, 1 exposure, 454 macros
03:03:36
03:03:38 Concurrency: 4 threads (target='dev')
03:03:38
03:03:39 Building catalog
03:03:49 Catalog written to /opt/airflow/dbt/exam_tech_bigdata/target/catalog.json
airflow@be1b7f3f285a:/opt/airflow/dbt/exam_tech_bigdata$
```

Fig1.6. d : Création du catalog dans le dossier target

```
dbt docs serve --port 8081 --host 0.0.0.0 (2)
```

La seconde (2) permet de se connecter en local via l'url suivante :

<http://localhost:8081>

(Voir le README.md pour ces commandes au point 10)

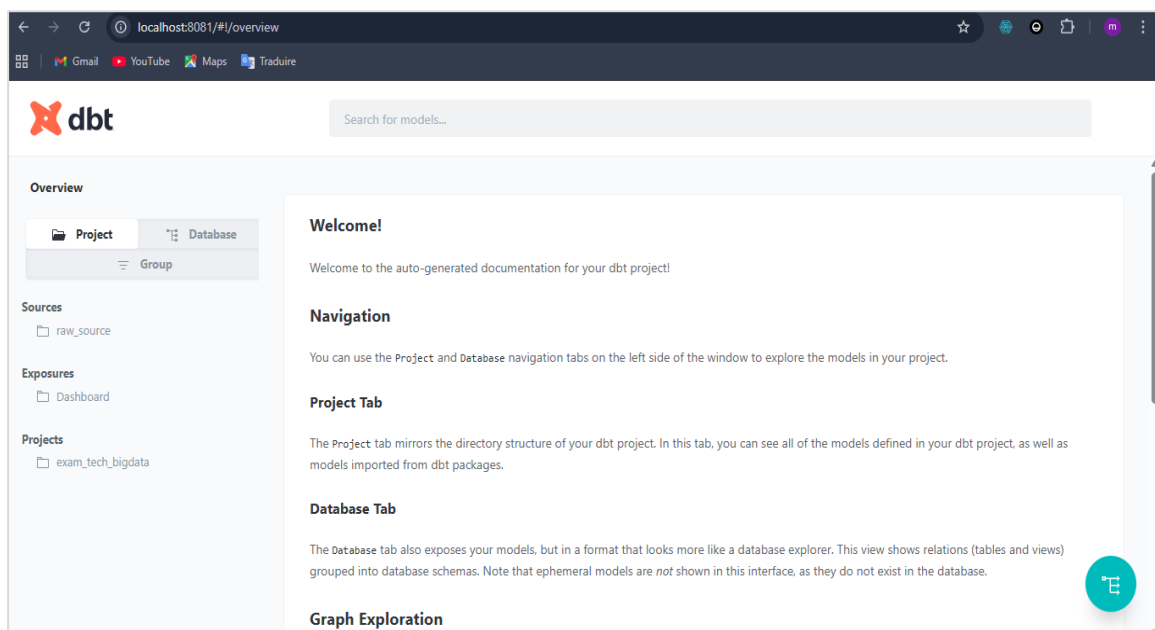


Fig1.6. e.1 : Accès à dbt en local

Ici on peut bien voir que l'on peut afficher notre base de données en cliquant sur « **Database** » ou voir la structure de notre projet ainsi que les relations et résultats attendus. De plus, à partir de là, vu la structure du schéma de [marts/obt_sales.sql](#) incluant une exposition du Dashboard, on peut accéder directement à la vue du tableau PowerBI par ici en cliquant sur sur « **Dashboard** » dans « **Exposures** » et en cliquant sur « **View this exposure** » (Il contient l'url de partage du tableau PowerBI).

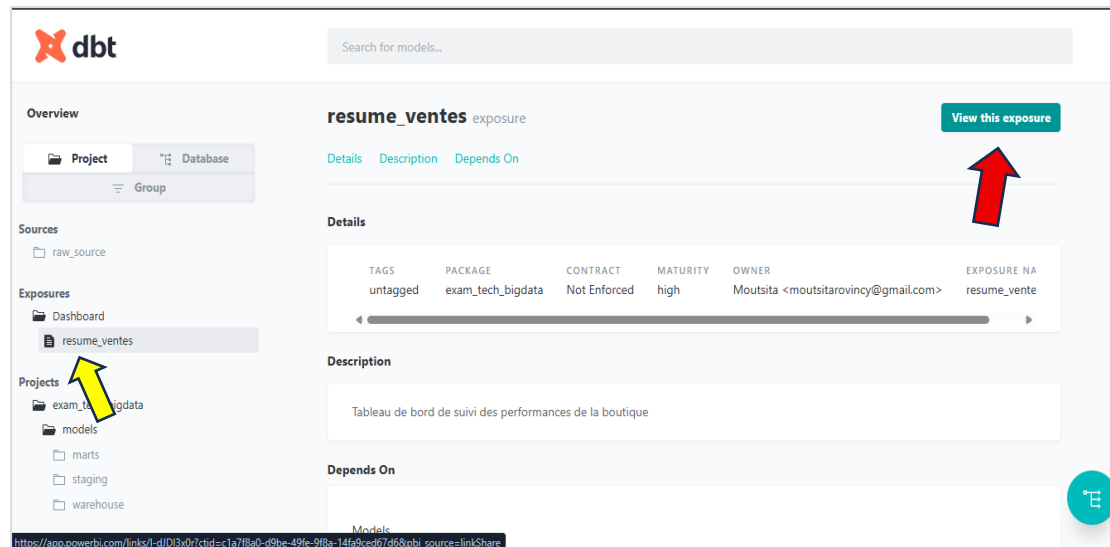


Fig1.6. e.2 : Illustration de comment accéder au résultat du tableau

La flèche en jaune indique le dossier et celle en rouge le lien du Dashboard.

Enfin, toujours dans le server dbt, on peut afficher ou voir les liaisons qui existent entre les parties de models, à savoir l'origine de la donnée, les transformations subies et le résultat final.

Pour le voir, il suffit de cliquer sur la partie source dans dbt serve et cliquer sur « **raw_source** » (Suivre les orientations des flèches).

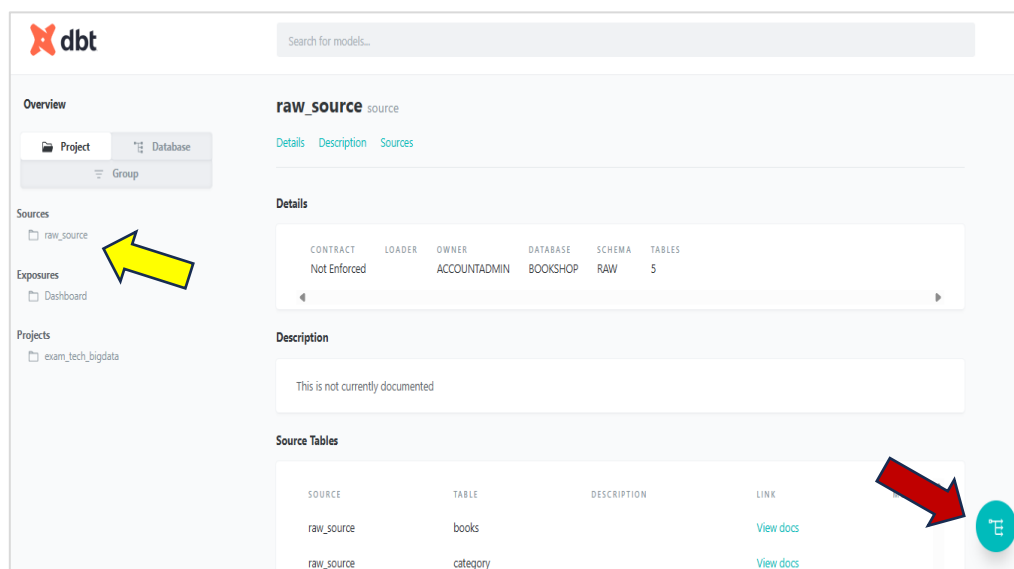


Fig1.6. f.1 : Illustration du trafic de transformation des données

Et cliquer enfin sur « [View Lineage Graph](#) », il renvoi le résultat ci-dessous :



Fig1.6. f.2 : Illustration du trafic de transformation des données

Etape 7 : Airflow, l'orchestration

En effet de toutes les commandes précédentes de DBT, il résulte que pipeline est opérationnel.

Pour ce faire, avant de lancer l'orchestration des pipelines, il faut d'abord créer un fichier config.py contenant nos identifiants Snowflake, respectivement dans les dossiers « [dags](#) » et « [app_admin](#) » (voir le point 11 du README.md).

- Lancement du dag d'ingestion de données de postgres vers snowflake

Elle précède, **l'étape 6** car c'est elle qui permet la migration de nos données de la base de données vers le cloud.

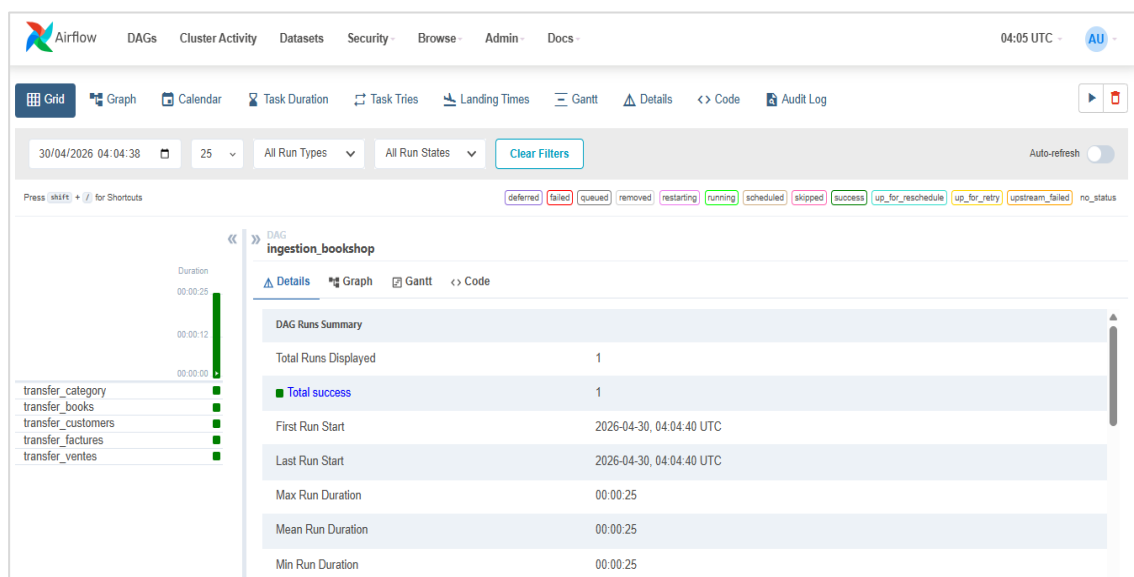


Fig1.7. a.1 : Illustration de la réussite de l'ingestion des données

Les graphes qui sont créés sont représentés comme suit :

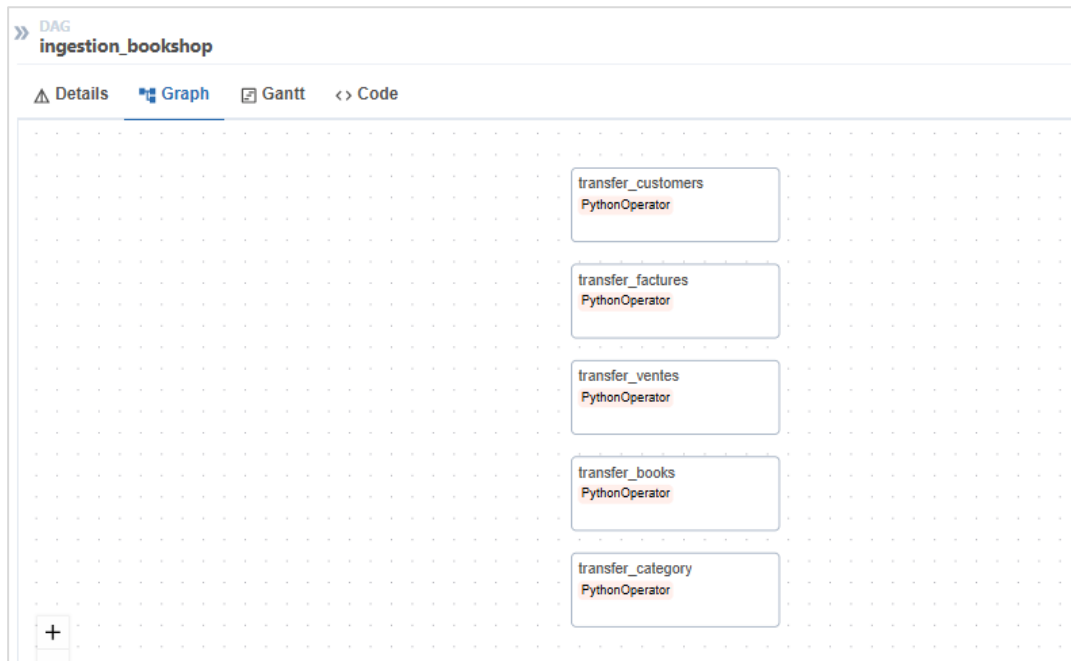


Fig1.7. a.2 : Représentation des graphes de l'ingestion des données

- Lancement du dag du pipeline_bookshoop

En effet le pipeline s'est exécuté sans erreur, les deux erreurs qui s'affichent sont le fait que nous avons eu a lancé le server via sa commande mentionnée ci-haut :

```
Dbt docs serve --port 8081 --host 0.0.0.0
```

Engendrant ainsi un conflit, nous avons dû, elle s'est exécutée sans souci

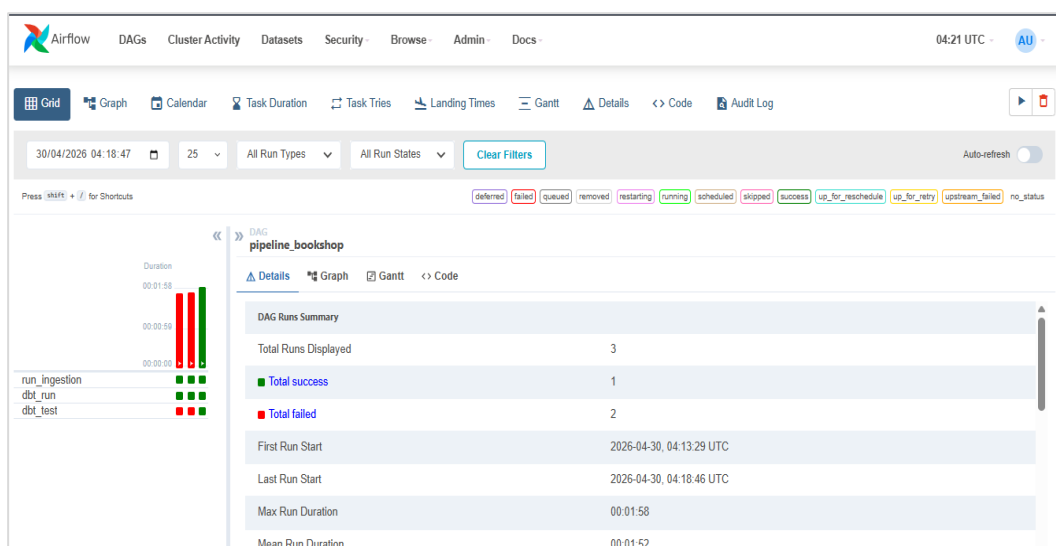


Fig1.7.b.1 : Représentation des graphes de l'ingestion des données

Voici la source de l'échec au lancement de notre pipeline, nous avons dû l'arrêter de manière volontaire (Ctrl + C) en le forçant et afin que le test s'exécute, d'où le passage au vert dans la fig1.7.b.1 ou nous avons réexécutés le pipeline ci-dessus :

```
airflow@belb7f3f285a: /opt/airflow/dbt/exam_tech_bigdata$ dbt docs serve --port 8081 --host 0.0.0.0
03:09:24 Running with dbt=1.8.7
Serving docs at 8081
To access from your browser, navigate to: http://localhost:8081

Press Ctrl+C to exit.
172.20.0.1 - - [30/Apr/2026 03:09:38] "GET / HTTP/1.1" 200 -
172.20.0.1 - - [30/Apr/2026 03:09:39] "GET /manifest.json?cb=1777518579695 HTTP/1.1" 200 -
172.20.0.1 - - [30/Apr/2026 03:09:39] "GET /catalog.json?cb=1777518579695 HTTP/1.1" 200 -
172.20.0.1 - - [30/Apr/2026 03:09:39] code 404, message File not found
172.20.0.1 - - [30/Apr/2026 03:09:39] "GET /%7B%7B%20getIcon(item.type,%20'on')%20%7D%7D HTTP/1.1" 404 -
172.20.0.1 - - [30/Apr/2026 03:09:39] code 404, message File not found
172.20.0.1 - - [30/Apr/2026 03:09:39] "GET /%7B%7B%20getIcon(item.type,%20'off')%20%7D%7D HTTP/1.1" 404 -
172.20.0.1 - - [30/Apr/2026 03:09:40] code 404, message File not found
172.20.0.1 - - [30/Apr/2026 03:09:40] "GET /%7Brequire('./assets/favicons/favicon.ico')%7D HTTP/1.1" 404 -
^C04:17:08 Encountered an error:

04:17:08 Traceback (most recent call last):
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/cli/requires.py", line 138, in wrapper
    result, success = func(*args, **kwargs)
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/cli/requires.py", line 101, in wrapper
    return func(*args, **kwargs)
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/cli/requires.py", line 218, in wrapper
    return func(*args, **kwargs)
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/cli/requires.py", line 247, in wrapper
    return func(*args, **kwargs)
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/cli/requires.py", line 294, in wrapper
    return func(*args, **kwargs)
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/cli/main.py", line 303, in docs_serve
    results = task.run()
  File "/home/airflow/.local/lib/python3.8/site-packages/dbt/task/docs/serve.py", line 29, in run
    httpd.serve_forever()
  File "/usr/local/lib/python3.8/socketserver.py", line 232, in serve_forever
    ready = selector.select(poll_interval)
  File "/usr/local/lib/python3.8/selectors.py", line 415, in select
    fd_event_list = self._selector.poll(timeout)
KeyboardInterrupt

^C
Aborted!
^C
airflow@belb7f3f285a: /opt/airflow/dbt/exam_tech_bigdata$ |
```

Fig1.7.b.2 : Arrêt forcé du server DBT local pour redonner la main au pipeline

Et le graphe qui montre l'ordre d'exécution du pipeline, de l'ingestion à la compilation jusqu'au test final.

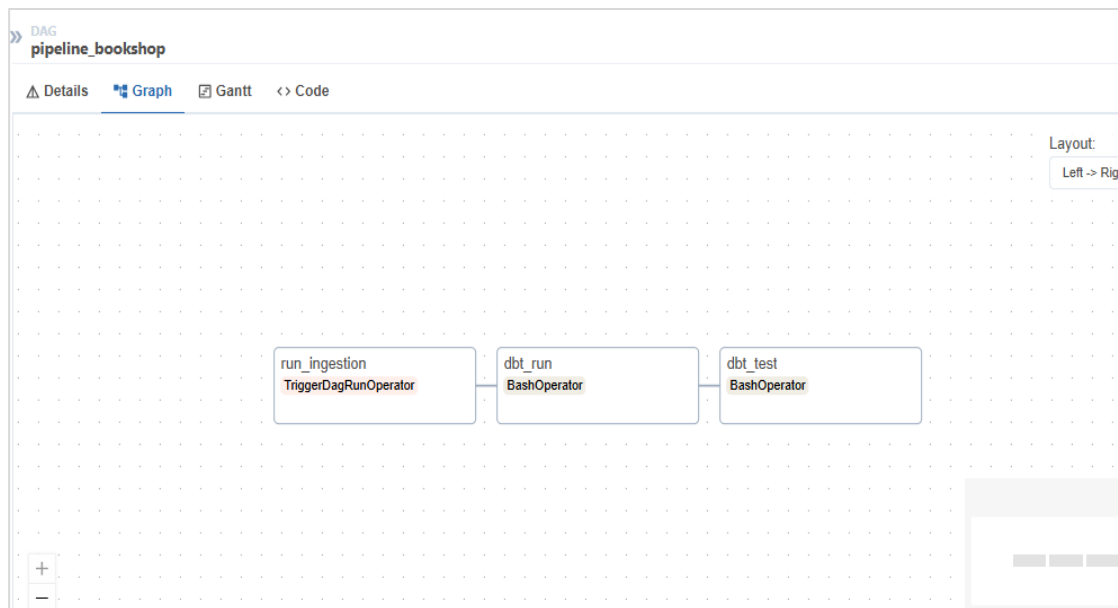


Fig1.7. b.3 : Représentation des graphes de liaison du pipeline_bookshop

Etape 8 : Snowflake et streamlit

Une fois l'ingestion des données effectuée, elles s'affichent automatiquement dans le cloud de snowflake.

```

1  USE DATABASE BOOKSHOP;
2
3  -- DROP DATABASE IF EXISTS BOOKSHOP;
4
5  USE SCHEMA RAW;
6
7  SELECT * FROM category;
8  SELECT * FROM customers;
9  SELECT * FROM factures;
  
```

ID	CODE	FIRST_NAME	LAST_NAME	CREATED_AT
31	C031	Estelle	Dossou	2026-04-29 22:14:27.908
32	C032	Patrick	Lissouba	2026-04-29 22:14:27.908
33	C033	Aline	Houngbedji	2026-04-29 22:14:27.908
34	C034	Serge	Ngoma	2026-04-29 22:14:27.908
35	C035	Mireille	Boukaka	2026-04-29 22:14:27.908

Fig1.8. a : Illustration de la requête qui affiche les clients dans snowflake

La requête de l'image illustre l'exécution de cette commande, elle affiche les données de postgres contenu dans db_init.

L'application streamlit est accessible via l'url suivante :

<http://localhost:8501>

Elle permet d'insérer les données directement dans la base de données postgres, comme nous l'illustrons sur les captures suivantes :

Navigation

Choisir la table RAW

customers

Mode

☒ RAW (Postgres)
 ☐ MARTS (Snowflake)

Insertion dans raw.customers

Code *

RZT013

Prénom *

Salif

Nom *

KEITA

Enregistrer

Données RAW

	id	code	first_name	last_name	created_at
0	36	RZT013	Salif	KEITA	2026-04-30 05:19:24
1	35	C035	Mireille	Boukaka	2026-04-29 22:14:27
2	34	C034	Serge	Ngoma	2026-04-29 22:14:27

Fig1.8. b : Insertion des données dans la BD postgres via l'app streamlit

Et de là, on relance dans airflow le dag d'ingestion afin que les données se mettent à jour et migrent vers snowflake :

Workspaces Databases + Q

postgres_diagram_bigdata_m2.sql requetes.sql

My Workspace exam_tech_bigdata requetes.sql

ACCOUNTADMIN COMPUTE_WH (X-Small) BOOKSHOP RAW Share

```

1 USE DATABASE BOOKSHOP;
2
3 -- DROP DATABASE IF EXISTS BOOKSHOP;
4
5 USE SCHEMA RAW;
6
7 SELECT * FROM category;
8 SELECT * FROM customers;
9 SELECT * FROM factures;
10 SELECT * FROM ...

```

Results (just now)

Table Chart 36 rows 175ms

#	ID	CODE	FIRST_NAME	LAST_NAME	CREATED_AT
32	32	C032	Patrick	Lissouba	2026-04-29 22:14:27.908
33	33	C033	Aline	Houngbedji	2026-04-29 22:14:27.908
34	34	C034	Serge	Ngoma	2026-04-29 22:14:27.908
35	35	C035	Mireille	Boukaka	2026-04-29 22:14:27.908
36	36	RZT013	Salif	KEITA	2026-04-30 05:19:24.957

Fig1.8. c : Migration des données insérées vers snowflake après relance du dag d'ingestion

Ainsi, on constate que nos données migrent bien de postgres vers snowflake via le dag d'ingestion de airflow.

Étape 9 : PowerBI

Comme mentionné dans l'étape 6, il suffisait de cliquer sur « [View this exposure](#) » pour avoir accès à l'image du tableau montrant notre travail.

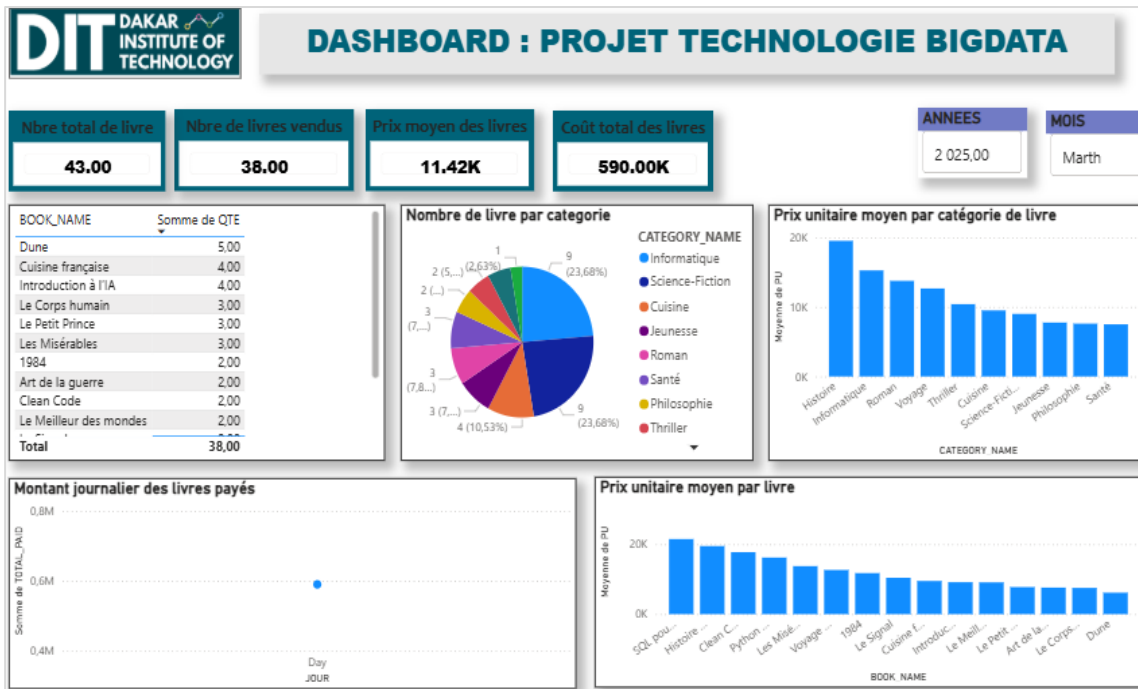


Fig : Illustration de resume_ventes qui tire ses données de marts/obt_sales.sql

Elle affiche l'ensemble des données attendus, cependant il n'affiche que des données d'un seul mois car lors de l'insertion des données dans les tables, nous avons mis la date comme étant celle en cours.

CONCLUSION

En définitive, La réalisation de ce projet a été riche d'enseignement car elle nous a permis de consulter à la fois le volet technique et savoir quand et le pourquoi une implémentation ne marche pas à travers les logs ; elle nous a permis de prendre connaissance sur certaines technologies qui sont nouvelles pour nous comme Postgres, snowflake ainsi que DBT, d'approfondir et asseoir certaines notions de Git et Docker ; de constater à la fois les divergences et les similitudes avec le langage MySQL pour la partie data, dont la syntaxe de base est quasi identique pour tous et d'être mis en confiance pour le lancement de scripts d'ingestion et du pipeline des données ; par la même mieux cerner la partie transformation que gère DBT par la construction des scripts SQL de models et de schémas d'implémentation de chaque partie du traitement, comprendre la complexité qui régit DBT, permettant une migration des données sans moindre effort et enfin afficher sous forme de graphes le résultats attendus par la partie business avec PowerBI, afin de répondre aux mieux aux besoins des utilisateurs de la donnée.

BIBLIOGRAPHIE ET WEBGRAPHIE

- <https://hub.docker.com/> : Pour la recherche des images appropriées pour faire tourner le projet dans des containers
- <https://github.com/> : Pour le partage et versioning du projet, afin que nous soyons tous à jour.
- <https://gemini.google.com/> et <https://chatgpt.com/> : pour nous aider à la construction de l'image de Airflow, contrairement aux autres, elle a été plus difficile à construire car ne retrouvant pas de guide comme dans les autres images.
- Cours « Gestion de code source avec Git/GitHub » de Diop Madicke, DIT
- Cours « Bases de données SQL » de Dr. Cheikh Ibrahima Fall DIOP, DIT
- Cours « Les Commandes Docker » de Diop Madicke, DIT
- <https://projector-video-pdf-converter.datacamp.com/39337/chapter3.pdf> : Cours en qui explique en détails la syntaxe DBT et commandes, de Susan Sun, freelance Data scientis
- Memento « Git & GitLab CheatSheet»
- Mémento «CheatSheet Docker »
- <https://daminoux.fr/les-notions-de-base-de-postgresql/> : les bases de postgreSQL
- <https://medium.com/@likkilaxminarayana/17-dbt-yaml-files-explained-5f3c0ddd906f> : Tout sur DBT, bien que le site soit en anglais, des images illustratifs de la structure aide facilement à la compréhension
- <https://datafinder.ru/files/downloads/01/The-Ultimate-Guide-to-dbt.pdf> : cours de DBT 101 by Maria Eugenio, illustre tout le processus de transformation de la donnée, est en anglais